ASSIGNMENT 2

Large Language Models, Chatbot Architecture and Vector Databases

---

A self-assessment and a research study on building LLM-based conversational systems and the retrieval infrastructure behind them.

Lakshin Khurana

October 19, 2026

CONTENTS

1 Question 1: Self-Assessment **1**
1.1 My Rating 1
1.2 Why B and Not A 1
1.3 Conceptual Evidence: Relative Self-Attention in My Music Transformer Project 2

2 Question 2: Architecture of an LLM-Based Chatbot **4**
2.1 Conceptual Understanding 4
2.2 Problem Statement 4
2.3 Proposed Solution 4

3 Question 3: Vector Databases **10**
3.1 Conceptual Understanding 10
3.2 Problem Statement 10
3.3 Proposed Solution 12

4 Conclusion **15**

References **15**

1. QUESTION 1: SELF-ASSESSMENT

1.1 My Rating

The scale is: *A* = can code independently, *B* = can code under supervision, *C* = little or no understanding.

TABLE 1. Self-assessment by area.

Area	Rating	Basis for the rating
Large Language Models	B	I understand the transformer, tokenization, prompting, RAG and fine-tuning, and can build working pipelines with guidance and reference material.
Deep Learning	B	I understand backpropagation, attention, normalization and training dynamics, and have implemented sequence models, though not always from a blank file.
Artificial Intelligence	B	I know the broader field (search, representation, agents, retrieval) at a conceptual level and apply it in projects.
Machine Learning	B	I understand the standard supervised workflow, evaluation and overfitting, and can implement solutions with some support.

1.2 Why B and Not A

Many of my everyday processes involve the use of AI tools, such as task delegation to agents. As I finish the work for them they can't manage, I'm working on parts that they can handle. This means that I'm productive, but in addition it also meant that I was not able to reliably create a in an empty editor without help. Implementation of a non-trivial deep learning system which is correct and optimized.

I don't think that C is the right rating either. When I read code and/or results carefully, I am able to detect conceptual errors, to reason about edge cases, and to use metrics to determine what needs to change in a system to make it work. This is B: I can conceptualize more than I can implement quickly.

> WHAT THIS MEANS IN PRACTICE
>
> I can research a problem, select a method to solve it, and provide a functional solution along with documentation and tools. The only difference between my current B and an A would be the ability to write complex model code from scratch without that, something I'm still working on.

1.3 Conceptual Evidence: Relative Self-Attention in My Music Transformer Project

To demonstrate the B relies on understanding and not just on tool use, I describe one concept that I used in a previous work, a decoder-only Music Transformer with *relative self-attention*. I will explain the idea, what I did and how my version differs from the original formulation below.

The problem with absolute positions

Standard self-attention [1] computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{1}$$

which is permutation-invariant. The input embeddings are supplemented with *absolute* positional encoding to provide order. This is not a good choice for music where the structure is largely relative: a note is significant because of its relationship to a previous beat, a repeated motif or the beginning of the bar. A motif played in position 40 and then again in position 400 is the same motif; however, absolute encodings show the two occurrences to the model in different ways.

Relative position information in the literature

Shaw et al. [2] made the attention score between positions i and j depend on the offset $j - i$. The Music Transformer of Huang et al. [3] adapts this idea to music with

$$e_{ij} = \frac{q_i k_j^\top + q_i r_{j-i}^\top}{\sqrt{D_h}}, \qquad S^{\text{rel}} = \text{skew}(QR^\top), \tag{2}$$

where $R \in \mathbb{R}^{L \times D_h}$ stores one learned embedding per relative distance. A direct implementation materializes a tensor of shape $L \times L \times D_h$, which needs $O(L^2 D)$ memory. The skewing step avoids this: Q is multiplied directly with $R^\top$, giving a matrix indexed by relative distance, which is converted to absolute indexing by padding a dummy column, reshaping, and slicing. This reduces the intermediate memory to $O(LD)$. A simpler alternative, used for example in T5 [5], replaces the embedding vectors with a single learned scalar bias per offset.

What I implemented

My implementation follows the scalar-bias approach. Each attention head owns a learnable vector $b \in \mathbb{R}^{2L-1}$ holding one scalar for every offset in $\{-(L-1), \ldots, L-1\}$, where L is the context length. For a sequence of length $T \leq L$, the logit between positions i and j is

$$e_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}} + b_{j-i} \quad (j \leq i), \qquad e_{ij} = -\infty \quad (j > i), \tag{3}$$

followed by a softmax over j. In code, this takes five steps:

(1) Build a $T \times T$ matrix of offsets $j - i$ from two index ranges.

(2) Clamp the offsets to $[-(L-1),\ L-1]$ and shift them by $L-1$ so that they are valid non-negative indices.
(3) Look up b with these indices to obtain a $T \times T$ bias matrix.
(4) Add this matrix to the scaled dot-product scores, broadcasting over the batch dimension.
(5) Apply the causal mask and the softmax.

How my version differs from the Music Transformer

TABLE 2. The original relative attention formulation compared with my simplified version.

	Music Transformer (Huang et al.)	My implementation
Relative term	$q_i r_{j-i}^{\top}$, a learned vector per offset	b_{j-i}, a learned scalar per offset
Depends on query content	Yes	No
Parameters per head	$L \times D_h$	$2L-1$
Extra memory	$O(LD)$ with skewing	One $T \times T$ index lookup
Skewing required	Yes	No

The bias is a scalar, so no $L \times L \times D_h$ tensor is created and there is no need to fix the bias. The only additional expense is an extra $T \times T$ index lookup, which is comparable to the cost of the attention matrix. The bias is the price of the expressiveness: the bias is not dependent on the query, so the model cannot consider a note four steps back relevant to one query and irrelevant to another. Each head can still learn its own preference over offsets, which is why I used one bias vector per head.

This was done because of the limitations on cloud resources like Google Colab, which did not allow for the original formulation. It provides a simplified description of the original method that was possible in those bounds.

What I learned from the project

- With absolute positional encodings, the model can only be generalized over a set of positions, but with relative positional encodings, the model can be generalized over a set of positions and over translation.
- Efficiency is a part of the architecture. The reason for the skewing trick is that relative embedding of vectors is costly, and my scalar bias is cheaper because I'm simplifying the model.
- Causal masking only uses half of the $2L-1$ bias entries, since only the non-positive offsets are used for gradient. A vector of length L would suffice.
- This was learned primarily by reading the papers and using tools to work through the shapes of the tensors. That's why I give myself a B: I'm better at the why than at writing each component without assistance.

2. QUESTION 2: ARCHITECTURE OF AN LLM-BASED CHATBOT

2.1 Conceptual Understanding

Beyond the model, a chatbot requires a number of other things

A large language model is a transformer-based next-token predictor that is trained on a very large dataset. There are three drawbacks to having one behind a chat box:

(1) The model is only aware of the information in the training data up to a certain point in time and does not have any private or recent information. It can also respond to questions beyond the scope of the task if not constrained.
(2) It generates fluent text, even if it is wrong, and does not have a concept of "I don't have a source for this."
(3) *Finite context:* Only the tokens in the context window are passed to each call, which means that any conversational memory has to be provided explicitly.

A production chatbot is not just a text box that's linked to an LLM. It is a system that is modeled, with a focus on providing knowledge, memory, safety and measurement, and requires explicit boundaries and a deliberate architecture.

Ways to provide an LLM knowledge

Most knowledge-based chatbots use RAG as their default option. Fine tuning is used to shape behaviour and complements it but is not a substitute for retrieval.

2.2 Problem Statement

> A B2B software company has approximately 2,000 pages of product documentation, API references and policy articles that are updated on a weekly basis. It desires a customer-support chatbot that (i) provides answers to questions *based on these documents*, (ii) references the source of each answer, (iii) refuses or escalates to a human when the documents do not contain an answer, (iv) supports multi-turn conversations, and (v) does not make one customer's private data visible to another.

2.3 Proposed Solution

High-level architecture

The design has an *offline* ingestion path that prepares knowledge and an *online* serving path that answers each message. Figure 1 summarizes the online path and its supporting components.

Component by component

Walkthrough of one request

(1) The user sends a message. Input guardrails strip or flag personal data, and identify attempts to override the system instructions.
(2) The query is reformulated as a standalone query with the help of the stored conversation memory.

TABLE 3. How to provide knowledge to an LLM.

Approach	Strength	Weakness	Best used when
Prompting with long context	Simplest, no infrastructure	Costly per call, degrades on very long inputs, no scaling to large corpora	Small, static knowledge
Fine-tuning	Changes style, format and behaviour	Poor at injecting facts, expensive to update, no citations	Tone, domain language, structured output
Retrieval-augmented generation (RAG) [4]	Fresh, citable, updatable without retraining	Quality bounded by retrieval; added latency and moving parts	Large or changing knowledge, need for citations

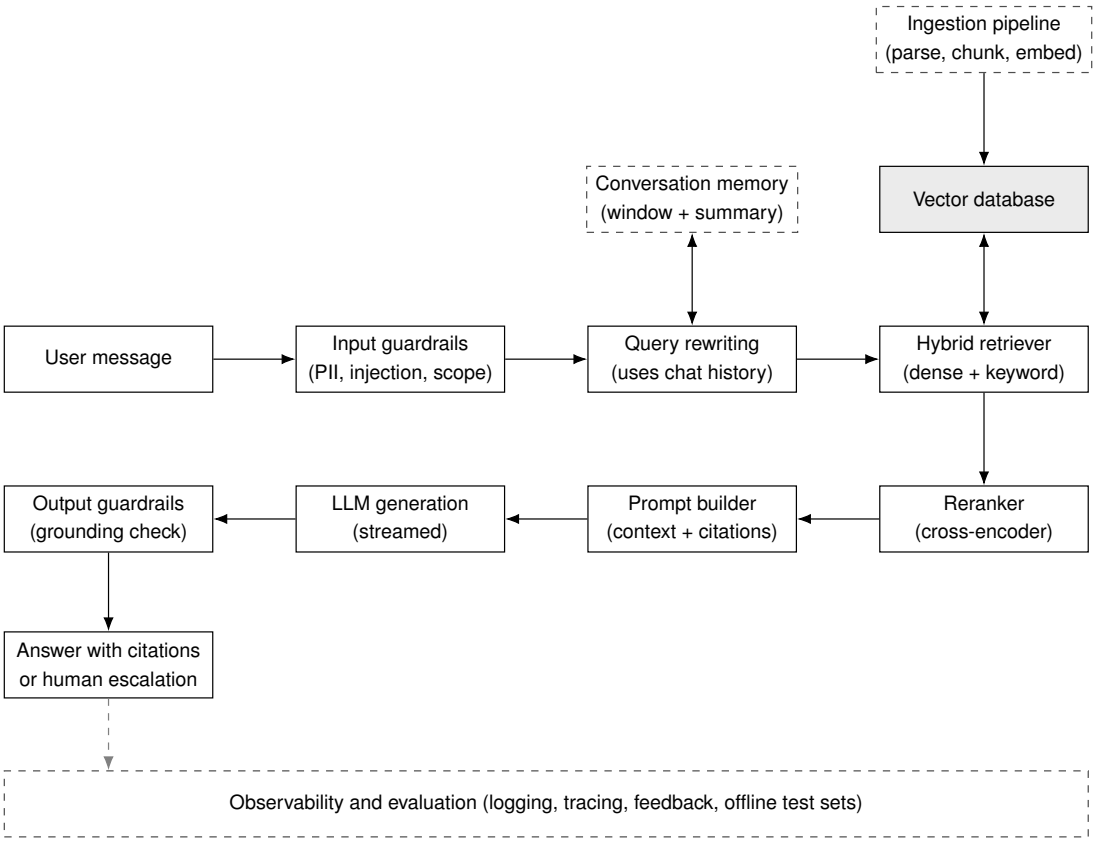


FIGURE 1. Online serving path of the proposed RAG chatbot, with its supporting stores.

TABLE 4. Key architectural components, their responsibilities and failure modes.

Component	Responsibility	Main failure mode to watch
Ingestion pipeline	Parse documents, clean them, split into chunks, attach metadata (tenant, version, access role), embed and upsert.	Bad parsing of tables or code; stale chunks after a document is updated.
Chunking	Split by document structure (headings, paragraphs) into pieces of a few hundred tokens with modest overlap.	Chunks too small lose context; too large dilute the embedding.
Embedding model	Map chunks and queries into the same vector space.	Domain mismatch; changing the model forces re-embedding everything.
Vector database	Store vectors and metadata; serve filtered approximate nearest-neighbour search.	Recall loss from aggressive approximation; missing filters leak data.
Query rewriting	Turn a follow-up such as "and what about the limit?" into a standalone query using the history.	Over-rewriting that changes the user's intent.
Hybrid retriever	Combine dense similarity with keyword (BM25-style) matching, merged by rank fusion.	Dense-only retrieval misses exact identifiers, error codes and names.
Reranker	Re-score the top candidates with a more accurate cross-encoder and keep the best few.	Added latency; must be limited to a small candidate set.
Prompt builder	Assemble system instructions, retrieved passages with source ids, history and the question.	Prompt injection through retrieved text; overflowing the context window.
LLM	Generate an answer using *only* the supplied context, in a defined format with citations.	Ignoring context and answering from parametric memory.
Guardrails	Screen inputs for PII and injection attempts; check that the output is supported by the context.	False refusals; checks too weak to catch unsupported claims.
Memory	Keep recent turns verbatim and summarize older ones.	Summaries that silently drop important constraints.
Observability and evaluation	Trace every request, collect feedback, and run regression tests.	Optimizing for demos instead of measured quality.

(3) The hybrid retriever retrieves, for instance, the top 30 candidate chunks from the vector database, based on the user's tenant and access role.
(4) These candidates are then re-scored by the reranker and the top 5–8 candidates are retained.
(5) The prompt builder combines the instructions, the selected passages (with a source id tag), the recent history and the question.
(6) The LLM produces the answer, with the instruction to provide passage ids and to state it does not know if the context is missing. The answer is sent in a stream to minimize perceived latency.
(7) Output guardrails ensure that the claims cited are backed by the passages. If they are not, the system reverts to a safe refusal or passes the conversation over to a human agent.

Evaluation

Improving a chatbot reliably requires measurement. Retrieval and generation should be evaluated separately, because a wrong answer can originate in either stage.

TABLE 5. Evaluation dimensions for a RAG chatbot.

Stage	Metrics	Question answered
Retrieval	Recall@k, MRR, context precision	Did the right passage reach the model?
Generation	Faithfulness (groundedness), answer relevance	Is the answer supported by the passages and on topic?
System	Latency (p50 and p95), cost per conversation, escalation rate, user feedback	Is it fast, affordable and actually useful?

A fixed set of representative questions with reference answers is run after every change to the chunking, embedding model, prompt or LLM, so that regressions are visible before release.

Design decisions and trade-offs

– *RAG is preferable to fine-tuning* since the documents change on a weekly basis, answers have to be citable, and it is much cheaper to update the index than it is to retrain.
– *Hybrid retrieval,* because support questions often include exact tokens, such as error codes or endpoint names. Pure semantic search does not handle these well.
– *There is a reranking stage* since the first-stage vector search has been optimized for speed and a cross-encoder applied to a small number of candidates can improve precision at a relatively low cost.
– *Refusal as a feature.* In the case of a support product, giving a confident but incorrect answer is worse than escalating, which is why the system chooses to

refrain from answering when the evidence is weak.

– *Control over cost and latency* involves response streaming, caching of frequently used queries, and the use of a smaller model for query rewriting with the larger model being reserved for the final generation.

3. QUESTION 3: VECTOR DATABASES

3.1 Conceptual Understanding

Embeddings and similarity

An embedding model takes various types of data, like text or images, and places them as points in a complex space. The goal is to group similar items close together. A vector database then keeps these points and their details, answering the question: which stored points are closest to a given query? The distance between them is calculated using a specific metric:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}, \qquad d_2(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2, \tag{4}$$

and the cosine similarity and the dot product are equivalent for normalized vectors.

Why not simply use a normal database?

An exact search is performed by comparing the query with all the stored vectors, and takes $O(Nd)$ time per query. That's on the order of 10^{10} multiplications per query, with ten million vectors of dimension 1024; that's too slow to use interactively. Vector databases thus use *approximate nearest neighbour* (ANN) indexes, where the recall is slightly reduced and controlled by a tunable parameter, but the latency is much lower.

Core indexing ideas

What is the difference between a database and a library?

The index algorithms are given in the form of a library, such as FAISS [8]. A *database* provides what production needs: persistence, create/update/delete operations, *metadata filtering*, replication and sharding, access control, backups and an API. The filtering capability is often crucial in a RAG system as queries are not generally "nearest vectors overall" but rather "nearest vectors *belonging to this tenant, in the current document version, that this user is allowed to see*".

3.2 Problem Statement

> In Question 2, the support chatbot has grown to be a multi-tenant service for ~200 business customers.
>
> - *Scale:* about 10 million text chunks, embedded at 1024 dimensions.
> - All queries must be filtered by tenant, document version and user access role.
> - *Search quality:* hybrid retrieval (dense vectors and keyword matching).
> - *Performance:* 50 queries per second at peak, retrieval p95 latency less than 100 ms.
> - *Team and budget:* two engineers, limited infrastructure budget, and the requirement to keep customer data in the company's cloud account.

Sizing the problem

There are no indexes overhead, so a set of 10 million float32 vectors of dimension 1024 requires $10^7 \times 1024 \times 4$ bytes ≈ 41 GB of raw storage. This requirement is crucial

TABLE 6. Common ANN and compression techniques.

Technique	Idea	Strength	Cost
HNSW [6]	A layered proximity graph; search goes from a sparse top layer to a dense bottom layer.	Excellent recall and speed.	High memory; slower to build.
IVF	Cluster vectors; search only the nearest few clusters.	Lower memory, fast build.	Recall depends on number of probed clusters.
Product quantization [7]	Compress vectors into short codes of sub-vector centroids.	Large memory savings.	Lossy; reduces accuracy.
Scalar / binary quantization	Store each dimension in 8 bits, or in 1 bit.	Large memory savings with small recall loss when combined with rescoring.	Needs a rescoring step for best quality.

because the working set should reside in RAM for quick search, and is lowered by lower precision, as seen in figure 2.

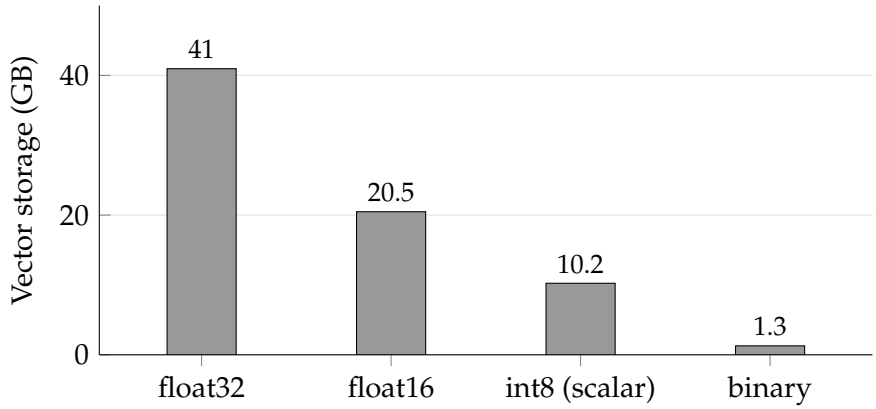


FIGURE 2. Approximate raw storage for 10^7 vectors at 1024 dimensions under different precisions (excluding index overhead).

3.3 Proposed Solution

Candidates considered

Decision and justification

> SELECTED: QDRANT
> I think Qdrant makes the most sense for this setup.

(1) Self hosting it inside the company cloud keeps everything private and the footprint stays small enough that two engineers could handle it.
(2) The filtered search part stands out because tenant isolation has to work on every query. Payload filters run during the graph traversal with indexed fields instead of trimming a list after the fact. That matters when the filter is strict and would otherwise leave almost nothing.
(3) Scalar quantization brings the vectors down from around forty one gigabytes to something closer to ten which fits the budget and still allows the original vectors on disk for rescoring the top hits.
(4) Hybrid search comes in too with sparse vectors next to the dense ones so keyword matches and semantic ones can combine without switching systems.
(5) Collections can be sharded later if the data grows which gives a path forward without starting over.

Proposed configuration

Validation plan

I am not totally sure how the rescoring count should be set if quantization starts hurting recall too much. A labelled set of questions with known relevant chunks would help check recall at ten against exact search as the baseline. Then the search

TABLE 7. Comparison of candidate vector databases against the stated problem.

Option	Strengths	Limitations for this problem	Verdict
FAISS	Very fast, flexible index library.	Not a database: no native filtering, persistence or CRUD.	Rejected
Chroma	Very easy for prototypes.	Less suited to heavy multi-tenant production load at this scale.	Prototype only
pgvector	Lives inside PostgreSQL; transactional and familiar.	Filtered ANN search at 10M vectors and 50 QPS needs careful tuning.	Strong runner-up
Pinecone	Fully managed, minimal operations.	Data leaves our cloud account; recurring cost grows with scale.	Rejected on privacy
Weaviate	Built-in hybrid search and modules.	Heavier to operate and tune.	Viable
Milvus	Very high scalability, rich index choices.	Complex distributed deployment for a two-person team.	Overbuilt
Qdrant	Payload-indexed filtering inside the HNSW search, hybrid support, quantization, simple deployment.	Smaller ecosystem than the largest players.	*Selected*

TABLE 8. Proposed Qdrant configuration for the multi-tenant knowledge assistant.

Aspect	Choice	Rationale
Index	The index would probably be HNSW tuned with moderate connectivity and a higher build depth	to hold recall steady under the tenant filters.
Distance metric	Cosine distance lines up	with how the embeddings were trained.
Payload indexes	Payload indexes on tenant id document version and access role	keep the filtering fast and the isolation enforced.
Tenancy model	One shared collection partitioned by the tenant field	avoids creating hundreds of tiny collections and the overhead that comes with them.
Hybrid search	Rank fusion handles the dense and sparse results together.	

depth gets adjusted until p95 latency stays under one hundred milliseconds even with the filters in a load test. If that still falls short the compression choice might need another look.

When I would choose differently

- *Under about one million vectors and an existing PostgreSQL stack:* `pgvector`, to avoid running another system.
- *No appetite for operations and no data residency constraint:* a managed service such as Pinecone.
- *Billions of vectors and a dedicated platform team:* Milvus, for its distributed scalability.

4. CONCLUSION

Across the three questions, one theme recurs: the model is only one part of a working AI system. Question 1 gives my honest rating, a B with conceptual grounding that I demonstrated through relative self-attention in my Music Transformer project. Question 2 shows that a reliable chatbot depends on retrieval, grounding, guardrails and evaluation as much as on the language model. Question 3 shows that the choice of retrieval layer is an engineering decision shaped by scale, filtering, privacy and team capacity, and that the right vector database is the one that fits the problem rather than the most popular one.

REFERENCES

[1] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] P. Shaw, J. Uszkoreit and A. Vaswani, "Self-Attention with Relative Position Representations," *NAACL-HLT*, 2018.

[3] C.-Z. A. Huang et al., "Music Transformer: Generating Music with Long-Term Structure," *International Conference on Learning Representations (ICLR)*, 2019.

[4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

[5] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, 2020.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[7] H. Jégou, M. Douze and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.

[8] J. Johnson, M. Douze and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, 2019.